AER1217 Final Project: Drone Racing

Yuhang Wang 1005048832
Andres Cervera Rozo 1005469217
Mohsen Diraneyya 1006462604

April 28, 2024

1 Introduction

1.1 Overview

Research and advancements in robotics and autonomous algorithms in recent years have allowed for autonomy in many tasks that humans had previously performed. Drone Racing is one of the key fields in which these advancements have been demonstrated. Although Drone Racing [1] is currently a sport where human pilots compete to fly quadrotors through complex tracks, it has been demonstrated that the autonomous flying of quadrotors could outcompete these human pilots.

The goal of this project is to develop a path planning algorithm for the time-optimal trajectory of a specific track and augmented with the Crazyflie nano-quadrotor PyBullet [2] environment for simulation, and CrazySwarm [3] controller command interface to interact with the hardware. This project is completed to compete in the drone racing competition of the AER1217 Course Final Project.

1.2 Competition Setup

The Drone Racing Competition occurs in the University of Toronto Myhal VICON room. This room is equipped with a VICON optical motion tracking system, which is used to mark the drone state and the map setup. These measurements are then injected with a uniform noise of $\pm 20cm$ applied to the location of the obstacles. These measurements are then used to estimate the drone's state. Along with this localization, the system also has a controller that interfaces low-level or high-level control commands.

The competition environment consists of four gates and five obstacles. All the gates are square entrances with 40 cm side lengths, surrounded by constrained tubes and installed at the same elevation. The obstacles are square pillars 12 cm in diameter. Figure 1 shows the environment setup as represented in the project handout.

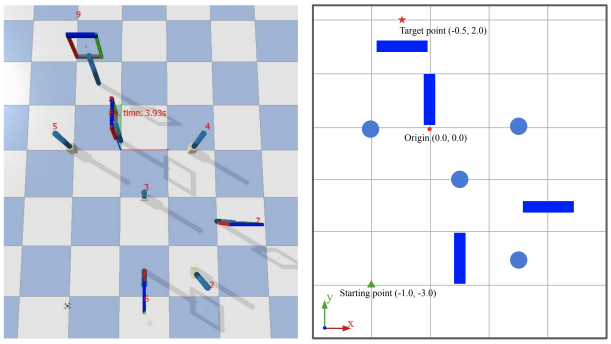

Figure 1: Competition Environment As Represented in the Project Handout

1.3 Approach and Planning Algorithm Selection

Since the racing track is set with the objective of flying the drone through the gates, and all the gates are at the same height, we limited the scope of the planning algorithm to the 2D optimal pathfinding. We used the high-level control commands to take off and land the drone from that height, and we used the low-level control commands to fly the drone through the course our algorithm found.

We investigated and implemented three different planning algorithms: RRT* [4], RRT* with Dubin path, and A* [5]. All three algorithms performed well in the PyBullet Simulator; however, we used the RRT* algorithm on the competition day since its performance on the test day was the best.

2 Implementation

2.1 Obstacle Generation

To plan a path within a predefined environment, it's crucial to represent the obstacles with some margin so that the planner can generate a path that does not collide/intersect with the obstacle in real-world testing. Since the obstacles and gates center in the environment are fixed in height, it's only necessary to plan and represent obstacles in 2D. The 4 cylindrical obstacles in the environment are $6cm$ in radius and have a uniform $20cm$ noise in their x and y positions. Therefore, these obstacles were represented by a square with a side length of $\mathbf{2} \times (\mathbf{noise} + \mathbf{radius_{obs}} + \mathbf{radius_{uav}} + \mathbf{buffer})$.

Additionally, we also defined obstacle boundary for the vertical gate segments which are red and green bars in the simulation, they are either on the left/right side or top/bottom side of the Gate depending on its orientation. Each Gate has two corresponding square obstacles that are $22cm$ away from its center; they all have a side length of $\mathbf{2} \times (\mathbf{radius_{uav}} + \mathbf{buffer})$. This is set up so that the planned path between two gates does not collide with any gates. This covers the gate center with obstacle boundaries but does not affect our planning algorithm since it's based on the gate's exit and entry point, which are explained in the following section. The environment has 12 square obstacle representations, 4 from the cylinders and 8 from the gates. Overall, we defined the obstacle with some buffer distance so that the drone stays away from objects even when cutting corners. For planning purpose, the obstacles are represented by an 3D array that defines the x and y boundaries (bottom left and top right coordinates) for each obstacle, Figure 3 shows the obstacles in blue boxes (pillars and gate limits).

2.2 Defining Trajectory Waypoints

We used a sampling-based algorithm for gate-to-gate planning (from the current gate exit to the next gate entry), but there are two ways of passing through a gate. Therefore, to generate waypoints for the overall trajectory, we determine the entry point for each Gate by overall optimal distance. A **Gate** object was defined with an entry point and exit point 40 cm away from the gate center based on the gate orientation defined in the YAML file. This defines the direction to go in and out of each Gate. A reversing function was also defined to flip the entry/exit point. The algorithm then creates binary masks for each Gate in the sequence, where a **0** means using predefined gate orientation for entry/exit point and **1** means applying the reverse function to the Gate. There are a total of $\mathbf{2^N}$ possible combinations for a **N** length gate sequence.

For the sequence **134213**, the combination of masks ranges from **000000** to **111111**. Then, a simplified overall distance is calculated as the distance from the start to the entry of the first Gate, plus the distance from the current gate exit to the next gate entry(iterated through all gates), plus the distance from the last gate exit to the final goal location. Note that this calculation is from the distances of the points and is not related to RRT* path planning. There are 64 entry/exit combinations for this specific sequence, and the one with minimal distance was used to generate waypoints and for discrete planning in the following section.

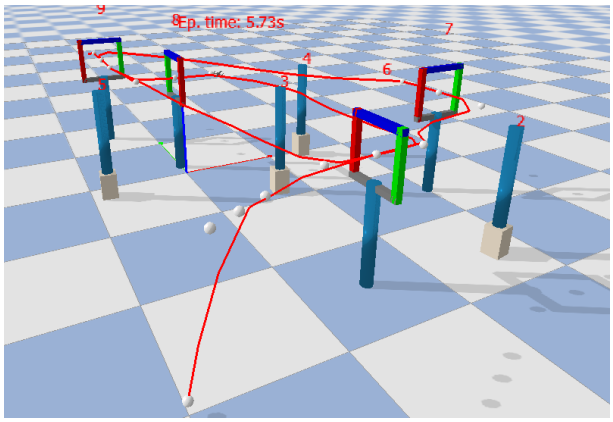

Figure 2: waypoint Generated and Smoothed Reference Positions

We also created a few extra waypoints from its starting position to the first gate entry to reduce drone flight time in testing. These extra waypoints were obtained from the result of our RRT* algorithm that plans from start to first entry. The first waypoint is set to have the takeoff height (0.1 meters) so that we can fit 3D reference points in later steps that guide the rover to the first entry point while ascending to an operation height of 1 meter. Figure 2 shows the waypoints as white spheres for the start position, intermediate points towards the first Gate, other gates in the planning sequence (entry, center and exit) and the final goal point.

2.3 Discrete Planning

Once we obtain waypoints connecting the starting point, gates in the sequence, and goal point, the next step is to fit a collision-free trajectory. The first part of this is to fit a 3D trajectory with an 8-degree polynomial fit for ascending.

The second part is that we iterate through the shortest entry/exit combination for the sequence to

- Perform RRT* from "current" exit to "next" entry to produce one segment's shortest straight path trajectory.
- This trajectory is then stacked with straight line segments (entry-center and/or center-exit) to complete the connection between segments.
- Perform 8-degree polynomial fit to linear segments.
- Use calculate the total distance of the sub-trajectory with a predefined speed parameter to find the corresponding time step and fitted xy positions in the trajectory.

By stacking the 3D trajectory with a 2D trajectory (at a height of 1 meter), we obtained smoothed reference positions with corresponding time steps. Figure 2 shows the smoothed reference positions as a red line. This can be used as positional commands that fly the UAV from the takeoff position (while ascending) to pass a sequence of paths (with optimal entry direction and trajectory) and finally arrive at the goal location. Sections 2.4 and 2.5 detail how the sampling-based algorithm determines the direct path and how that path is smoothed to a trajectory, respectively.

2.4 RRT*

RRT* was implemented as a sampling-based path planning algorithm to determine the most direct path between each Gate. RRT* is characterized by taking iterations of samples in the configuration space while continuously rewiring to ensure that each calculated path is the most efficient for a given location. Gaussian-based sampling was implemented rather than the typical sampling of points in the full map. More specifically, the start and end coordinates in 2D space were set as $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$,

respectively. Equations 1, 2, 3, and 4 define how these points are used to sample more useful locations, where the standard deviation and buffer values were determined through experimentation. Note that in the code implementation, the sampling equations are transposed concerning the x and y axes for more vertical than horizontal paths from a bird's eye view. Furthermore, to ensure that the growth of the network of nodes expanded towards the next Gate, a bias term of 20% was set.

$$m = \frac{y_2 - y_1}{x_2 - x_1} \tag{1}$$

$$c = y_1 - mx_1 \tag{2}$$

$$x_{\text{sample}} = \text{uniform}(x_1 - buffer, x_2 + buffer) \tag{3}$$

$$y_{\text{sample}} = mx_{\text{value}} + c + \text{normal}(0, \text{std\_dev}) \tag{4}$$

After picking a sample, rewiring of the RRT* node network ensured that the parent of the new node had a collision-free straight line path to the new node, and all nearby nodes were rewired to have the new node as a parent node if that would result in a smaller total path distance to the start point. After a set amount of iterations and rewiring, the algorithm would return a near-optimal path of a series of straight line paths that connect each path between gates as seen in Figure 3. As intended, the tree of nodes (black) is seen to expand specifically in the general direction of the next Gate, without sampling in unwanted areas, to result in the final path (red).

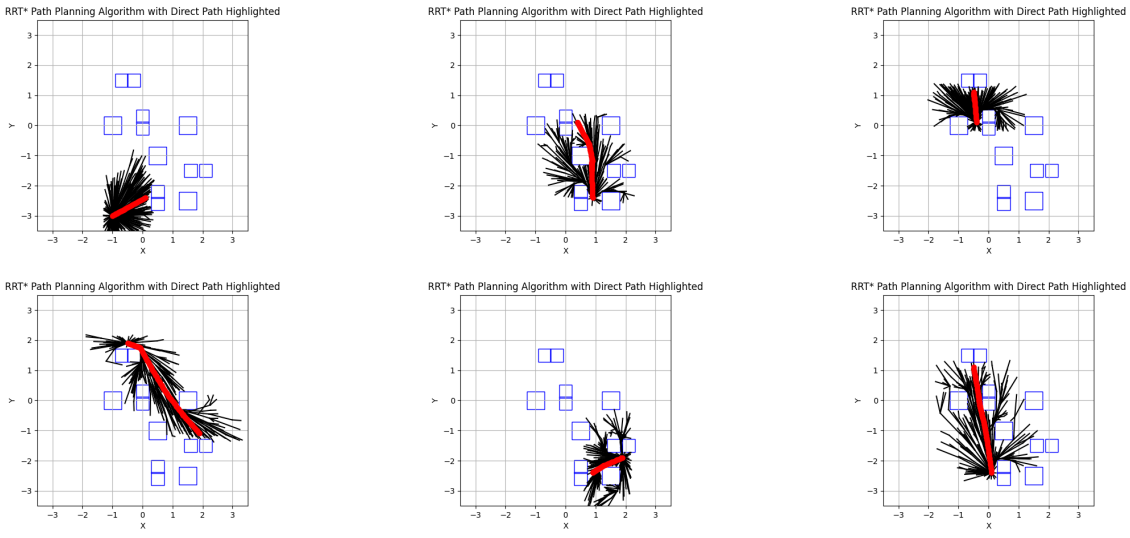

Figure 3: Intermediate Path Planning for Sequence 134214

2.5 Curved Trajectory Generation

Each segment path output from RRT* was made smooth with an 8-degree polynomial fit to allow for a stable and feasible flight. The straight-line segments were stacked with linearly spaced points inside the Gate to ensure a smooth transition between trajectory segments and free of sharp 90° turns. A global variable determined the number of points inside the gate region to allow flexibility in determining the importance of going straight through the Gate. Samples of the original and resulting curved trajectory segments may be seen in Figure 4.

The frequency of the controller was set by default to be 30 Hz. This indicated that the resulting xyz coordinates should consider the frequency limit when computing a desired location for a given time stamp. A global variable was set to indicate the drone's target velocity throughout the trip. By determining the

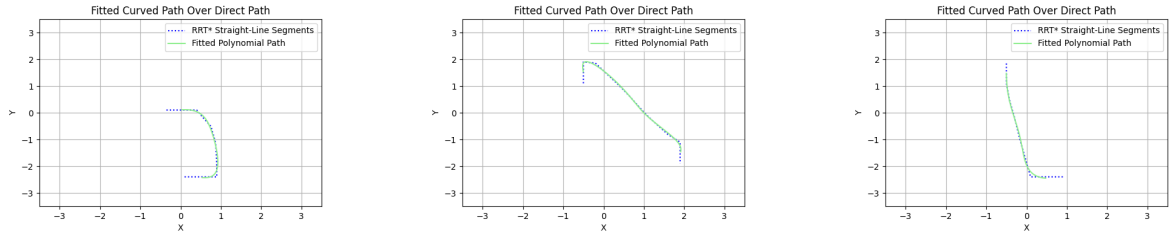

Figure 4: Samples Of Curved Polynomial Fits Over RRT* Outputs

length of each trajectory segment and leveraging the target velocity, coordinates were sampled from the curved polynomial equations at intervals that would coincide with the controller's frequency to ensure the desired velocity was reached. The resulting coordinates and time stamps for the full trip were stacked together and saved offline in a CSV file based on the desired gate order. Time is saved in the drone takeoff by computing the trajectory offline and accessing the CSV file at launch time.

3 Result

3.1 Simulation

All three planners' implementations on the PyPullet simulation environment were successful. We demonstrated that each implementation can complete any path track composed of arbitrary gate sequences within the competition map. Furthermore, we tested the limits of the maximum possible speed we can operate for different sequences through the full state control parameters of the waypoint time variable. Tuning these control parameters allowed for faster track completion; however, the controller lag was the bottleneck for the maximum achievable speed and completion time without crashes.

Using the PyPullet simulation of the drone using the RRT Star planning algorithm on the competition gate sequence of **134213** showed that a completion time of the track in 16.0 seconds was achievable.

3.2 Competition

The track flight completion time achieved in the simulation environment was not achieved during the testing phase with the hardware, possibly due to deviations in the drone parameters from the simulation and controller. These deviations could include the drone's inertial parameters, rotor damage or deformation, battery output power, or the DC motor parameters. Nonetheless, running the simulation was essential for debugging and fixing algorithm issues, tuning the planning and control parameters for different flight paths and sequences, and the different planning algorithms.

On the other hand, the testing sessions allowed us to test the limitations of the actual hardware, controller, and state estimation algorithms. They showed that there could be a shortcoming of the controller, especially at higher speeds, where small model deviations from the actual hardware could result in a significant drift over time. In those sessions, we determined the maximum achievable speed with our implementation. Furthermore, they were insightful for us to adjust our RRT star implementation to minimize the sharpness of the track curvature by adjusting the trajectory smoothing and polynomial fit algorithms to trade-off between the sharpness of the curvature, the path length and the resemblance of the final trajectory to the RRT star path planning trajectory output.

By making these adjustments and carefully optimizing the low-level controller and path-planning algorithm parameters based on the simulation and testing results, we were able to successfully complete the competition path **134213** in **18.4 Seconds**, achieving the **2nd Place** in this competition.

It is possible to improve upon this implementation by augmenting a robust controller that compensates for the hardware deviation from the model to achieve the same results achieved in the simulation environment. It may also be possible to achieve higher speeds by carefully optimizing an RRT start path planning that followed the Dubin path, thus ensuring that a smooth obstacle-free path is achieved with minimum curvature to allow for higher operation speed.

References

[1] Drone Racing League. Drl.

[2] et al. Erwin Coumans. Pybullet, 2022.

[3] James A. Preiss Wolfgang Hoenig and contributor. Crazyswarm, 2021.

[4] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *The international journal of robotics research*, 30(7):846–894, 2011.

[5] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.